

Taller 2

Tecnologías y herramientas: Python, Flask, SQLite, pytest, Postman

API externa: JSONPlaceholder

Objetivo general del taller

Construir una aplicación que permita comprender, mediante la práctica, la diferencia real entre pruebas unitarias y pruebas de integración, y entender por qué se usan los test doubles.

NOTA: No se trata de hacer que el código funcione, se trata de entender por qué funciona y por qué deja de funcionar.

Paso 0 – Preparación del entorno

Instalar dependencias:

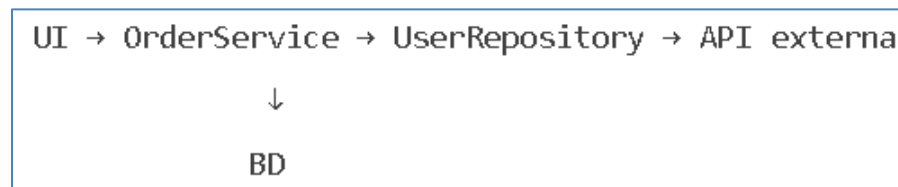
```
pip install -r requirements.txt
```

Evidencia: Enlace del repositorio en GitHub.

Paso 1 – Contexto de la aplicación

La aplicación permite crear órdenes. Cada orden pertenece a un usuario, pero los usuarios no se almacenan localmente: se obtienen desde un servicio externo. Las órdenes se guardan en una BD local.

Arquitectura conceptual:



Pregunta: ¿Qué parte del sistema no controlamos?

Paso 2 – Persistencia con SQLite

Se define una base de datos SQLite con la entidad Order.

Pregunta: ¿Por qué la base de datos es una dependencia?

Paso 3 – Repositorio (API externa) real de usuarios

Se implementa `JsonPlaceholderUserRepository`, que consulta la API pública `JsonPlaceholder` para obtener el email del usuario.

Verificar con Postman que existe email para el usuario con id 1.

<https://jsonplaceholder.typicode.com/users/1>

The screenshot shows a Postman interface with a GET request to `https://jsonplaceholder.typicode.com/users/1`. The response is a 200 OK status with a response time of 846 ms and a body size of 1.61 KB. The response body is displayed in JSON format, showing the details of a user with id 1.

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  }
}
```

Pregunta: ¿Quién garantiza que el usuario existe?

Paso 4 – Lógica de negocio

La función **create_order** recibe todas sus dependencias como parámetros: logger, notifier, repositorio de usuarios y base de datos. Se debe completar el código en **order_service.py**.

Pregunta: ¿La lógica sabe de dónde viene el usuario?

Paso 5 – Interfaz web funcional

Se construye una aplicación Flask que permite crear órdenes desde un formulario web. La aplicación usa el repositorio real de usuarios y una base de datos SQLite.

Se ejecuta con: `python app.py`



The screenshot shows a web application with a title "Create Order". Below the title, there are two input fields: "User ID:" and "Amount:". Below these fields is a button labeled "Create Order". Below the button, there is a section titled "Orders" which contains a list of two items: "user7@fake.local - 78 - CREATED" and "user5@fake.local - 8 - CREATED".

Comprobar que:

- se crean órdenes
- se consultan usuarios reales
- los datos persisten

Evidencia: Orden creada y datos persistidos.

Paso 6 – Test ‘normal’ con API real

Se crea un test que ejecuta `create_order` usando la API real. Aún no se usan test doubles para reemplazar la dependencia externa.

Se define `DummyLogger` y `NullNotifier` con métodos vacíos.

Completa el código del archivo **`pytest test_unit_real_api.py`**

Ejecuta el test unitario con: `pytest test_unit_real_api.py -v`

Completa el código del archivo **test_integration_real_api.py**

Ejecuta el test de integración con: `pytest test_integration_real_api.py -v`

Luego, ejecuta la app otra vez: `python app.py`

Evidencia: Salida del test unitario.

Evidencia: Salida del test de integración.

Pregunta: ¿Este test es siempre confiable? ¿Por qué?

Pregunta: ¿Cuál es la diferencia entre `test_unit_real_api` y `test_integration_real_api`?

Paso 7 – Romper la dependencia externa

Se modifica el repositorio real para lanzar una excepción.

En el archivo **user_repository.py** usa únicamente la clase que contiene el error "User service unavailable".

Ejecuta el test unitario con: `pytest test_unit_real_api.py -v`

Ejecuta el test de integración con: `pytest test_integration_real_api.py -v`

Luego, ejecuta la app otra vez: `python app.py`

Todo falla.

Evidencia: Salida del test unitario.

Evidencia: Salida del test de integración.

Evidencia: Ejecución de la app.

Pregunta: ¿Qué se rompió realmente?

Paso 8 – Introducir un Fake

Se implementa `FakeUserRepository`, que retorna emails ficticios. La aplicación vuelve a funcionar sin usar la API.

Cambia la app para usar FakeUserRepository (en **user_repository.py**). También debes modificar **app.py** para usar FakeUserRepository en lugar de JsonPlaceholderUserRepository().

Ejecuta la app otra vez: `python app.py`

Evidencia: App funcionando sin API.

Paso 9 – Test unitario real con Stub

En este paso se crea un test unitario verdadero. El StubUserRepository se define dentro del archivo del test y se inyecta en create_order. No se usa red ni servicios externos.

Completa el código en **test_unit_with_stub.py**.

Ejecuta: `pytest test_unit_with_stub.py -v`

Pregunta: ¿Por qué este sí es un test unitario?

Evidencia: Ejecución del test unitario.

Paso 10 – Test de integración sin API externa

Se ejecuta un test que integra lógica y base de datos, pero usa FakeUserRepository. No hay red, pero sigue siendo una integración interna.

Ejecuta: `pytest test_integration_with_fake.py -v`

Pregunta: ¿Por qué sigue siendo integración?